

Week 2 Day 5: Docker Compose로 회사형 서비스 아키텍처 로컬 재현하기

Overview

Day 5는 Compose 문법을 길게 외우는 날이 아니다. 1교시에서 Compose 기본 개념과 편의성을 정리한 뒤, 2~8교시는 Week 1 Day 4에서 본 회사형 서비스 아키텍처를 로컬 Compose template으로 직접 실행한다.

오늘의 핵심 질문은 다음과 같다.

회사 서비스에서 보던 frontend, gateway, API, DB, cache, queue, worker를 compose.yaml과 작은 API 앱으로 재현하면 실행, 검증, 장애 분석, 인수인계가 어떻게 쉬워지는가?

각 교시는 하나의 architecture folder를 사용한다. 학생은 compose.yaml을 읽고, docker compose config, up, ps, logs, curl, exec, down으로 실제 동작을 확인한다.

Learning Goals

- Compose가 긴 docker run 명령을 어떻게 재현 가능한 template으로 바꾸는지 설명한다.
- services, ports, environment, volumes, depends_on, service name DNS를 읽는다.
- 커머스 카탈로그, 백엔드 경계, 프론트엔드 설정, 메시징, 주문 이벤트, 폭주 트래픽, MSA preview 패턴을 실행한다.
- 각 template에서 연결 증거를 확인한다.
- down과 down -v의 차이를 data lifecycle 관점에서 설명한다.
- Week 3 MSA의 service boundary, dependency, failure propagation 질문을 만든다.

Lesson Index

교시	주제	Template	핵심 확인
1교시	Compose 기본 개념과 편 의성	공통 루프	config/up/ps/logs/down
2교시	쿠팡형 커머스 카탈로그	01-web-postgres	frontend, catalog API, products DB
3교시	당근형 백엔드 서비스 경계	02-web-postgres-admin	gateway, identity API, payment API, Adminer
4교시	토스형 프론트엔드 플랫폼	03-web-redis	frontend preview, config API, Redis cache
5교시	Gateway routing	04-nginx-reverse-proxy	/a/, /b/, upstream failure
6교시	카카오형 메시징/worker	05-queue-worker-db	HTTP producer, Redis queue, worker logs
7교시	API + PostgreSQL	06-api-postgrest	REST API response, DB init logs
8교시	MSA preview	07-frontend-gateway- api-db	frontend marker, /api/services, Week3 bridge

Practice Files

자료	용도
hands-on-lab.md	전체 실행 순서
labs/compose-architectures/01-web-postgres/	two-tier web + DB
labs/compose-architectures/02-web-postgres-admin/	DB Admin UI
labs/compose-architectures/03-web-redis/	cache backing service
labs/compose-architectures/04-nginx-reverse-proxy/	gateway/reverse proxy
labs/compose-architectures/05-queue-worker-db/	async queue/worker
labs/compose-architectures/06-api-postgres/	API + DB
labs/compose-architectures/07-frontend-gateway-api-db/	MSA preview
assets/day5-compose-architecture-lab-overview.png	전체 구조 인포그래픽
assets/day5-arch-*.png	각 Compose architecture의 network area/연결선/서비스 아이콘 구조도

Common Verification Loop

```
docker compose config
docker compose up -d
docker compose ps
docker compose logs --tail 80
```

Service type별 추가 확인:

Service type	확인 명령 예시
Web/frontend	curl -I http://localhost:<port>
API	curl -s http://localhost:<port>/<resource>
PostgreSQL	docker compose exec db psql ...
Redis	docker compose exec redis redis-cli GET ...
Worker	docker compose logs worker --tail 40
Gateway	public port로 접근, 내부 service port는 직접 열지 않음

주의할 점

위험	확인 지점
Compose config만 보고 성공으로 착각	up, ps, service-specific check까지 실행
wrong service name	container 내부에서는 localhost가 아니라 service name 사용
wrong port	host 접근은 published port, service 간 접근은 container port
stale volume	DB 초기 데이터가 예상과 다르면 volume 확인
down -v 남용	DB/cache volume 삭제 여부 확인
Admin UI 공개	host port 노출 목적과 종료 시점 확인

End-Of-Day Checklist

- [] 1교시 공통 Compose 검증 루프를 설명했다.
- [] 최소 4개 architecture template을 실제 실행했다.
- [] 각 template에서 연결 증거를 하나 이상 확인했다.
- [] service name, host port, container port를 구분했다.
- [] DB volume이 있는 template에서 down과 down -v 차이를 설명했다.
- [] reverse proxy 또는 gateway에서 외부 진입점과 내부 service를 구분했다.
- [] queue/worker 또는 API+DB template에서 logs/API/DB query를 함께 확인했다.
- [] Week 3 MSA로 가져갈 dependency/failure 질문을 작성했다.

Completion Definition

Day 5는 다음 문장을 증거와 함께 말할 수 있을 때 완료된다.

Compose template은 여러 container의 실행 조건을 파일로 남기고, service name/network/volume/port를 통해 자주 쓰는 architecture를 재현 가능하게 만든다.

Next Connection

Week 3에서는 MSA를 다룬다. Day 5에서 실행한 frontend, gateway, API, DB, cache, worker는 Week 3에서 service boundary, health check, deployment, Kubernetes manifest로 이어진다.